# University of Haifa

**Department of Information Systems**

**Software Engineering and Software Quality Assurance**

# Software Requirements Specification (SRS)

## Educational Ludo Game

**Based on IEEE Std 830-1998**

Assignment 2 – Software Requirements Specification

**Group Name:** Sharks

**Team Members:** Shafeek Saif , Rakan Ziyan , Mohamed Zerny

**Document Version:** 1.0

**Submission Date:** October 25, 2026

# Table of Contents

## 1. Introduction

### 1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the functional and non-functional requirements of the Educational Ludo Game system. The document describes the expected behavior, features, interfaces, and constraints of the system, based on the project requirements provided by the course staff.

This SRS serves as a reference for the development team, the course staff acting as the customer, and the software testing team. It establishes a clear foundation for system design, implementation, and verification, following the guidelines of IEEE Std 830-1998.

### 1.2 Scope

The Educational Ludo Game is a computer-based board game designed for students studying Software Engineering and Software Quality Assurance. The system combines traditional Ludo gameplay with educational multiple-choice questions related to the course material, providing an interactive and engaging learning experience.

The system supports two to four players and offers three difficulty levels: Easy, Medium, and Hard. Players can select their names and colors, roll dice, move their pieces across the board, and interact with special squares, including question, surprise, and protection squares. Each difficulty level introduces different board configurations and gameplay features.

The system also includes a game history feature that records completed games, player rankings, difficulty levels, and game duration. Additionally, an administrator can securely access a question management interface to add, edit, delete, and view educational questions.

The application will be developed in Java, following the project requirements provided by the course staff.

### 1.3 Definitions, Acronyms, and Abbreviations

| Term | Definition |
|---|---|
| SRS | Document specifying software requirements. |
| IEEE 830 | Standard providing guidelines for SRS documents. |
| Player | User who plays Ludo and answers questions. |
| Game Piece | Moveable token controlled by a player. |
| Game Board | Playing area containing tracks and squares. |
| Castle | Starting area holding a player's four pieces. |
| Shared Track | The common path followed by player's pieces around the board. |
| Home Path | The final, player-specific path leading to the finishing area. |
| Protected Square | A square where pieces cannot be captured by opponents. |
| Double Block | Two or more same-player pieces blocking a square. |
| Question Square | A square that triggers an educational multiple-choice question. |
| Question Bank | A collection of educational questions organized by difficulty level. |
| Surprise Square | A square that activates a randomly selected gameplay effect |
| Boost / UltraBoost | Special dice affecting movement according to difficulty level. |
| Game History | Stored results of completed games. |
| Administrator | Authorized user managing the question bank. |

| | |
|---|---|
| CSV | Comma-Separated Values file format. |
| MVC | Model-View-Controller software architecture. |
| FR / NFR | Functional Requirement / Non-Functional Requirement. |

### 1.4 References

The following sources were used to prepare this SRS:

1- IEEE. *IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications*, 1998.

2- University of Haifa. *Ludo Project – Group Project Description*, 2026.

3- University of Haifa. *Assignment 2 – SRS*, October 19, 2026.

4- Course Staff. *ANSWERS.xlsx – Student Questions and Staff Clarifications*.

5- Course Staff. *History-DataBase Template – Game.csv*.

6- Sharks Team. *Assignment 1 – Ludo Project: Initial Entities and Requirements*, 2026.

7- University of Haifa. *Practice Lesson 2 – Scrum Methodology & SRS*.

Course materials are available through the course website. Official project requirements and staff clarifications take precedence over team assumptions.

### 1.5 Document Overview and Organization

This SRS follows IEEE Std 830-1998 and consists of three main sections: **Introduction**, covering the document's purpose, scope, terminology, and references; **Overall Description**, presenting the system's functions, users, constraints, and assumptions; and **Specific Requirements**, defining detailed functional and non-functional requirements.

**Selected Structure and Justification**

Section 3 follows the combined **User Class and Feature-based structure (IEEE 830, Annex A.8)**. This approach was selected because the system has two distinct user classes: Players, who interact with gameplay features, and Administrators, who manage the question bank. Requirements are organized by features within each user class, improving clarity, traceability, and testability.

Alternative structures were considered less suitable. Organizing by difficulty level would repeat common gameplay requirements, while an object-based structure would be less effective for describing user interactions.

**Non-Applicable Sections**

Dedicated hardware, network communication, and site-specific installation requirements are not defined in the official project specification. Their applicability is addressed in the relevant sections of this SRS.

## 2. Overall Description

### 2.1 Product Perspective

The Educational Ludo Game is a standalone Java application developed for the Software Engineering and Software Quality Assurance course.

The system provides a graphical user interface for gameplay, educational questions, game history, and administrator-only question management. Completed game records are stored using the predefined CSV schema, while the question bank supports educational gameplay.

No external system integration or online services are required by the current project specification.

### 2.2 Product Functions

The Educational Ludo Game provides seven main functions:

1. **Game Setup:** Allows 2–4 players to select a difficulty level, enter their names, and choose unique colors.

2. **Board Configuration:** Generates the appropriate board, tracks, dice, and special squares based on the selected difficulty level.
3. **Turn and Dice Management:** Manages player turns, dice rolls, movement calculations, and additional turns according to game rules.
4. **Piece Movement:** Enforces movement rules, including leaving the castle, capturing opponents, protected squares, Double Blocks, and reaching the finishing area.
5. **Questions and Surprise Events:** Presents educational questions, adjusts movement based on answers, and activates random surprise effects.
6. **Game History:** Determines player rankings, saves completed game results in the required CSV format, and displays previous games.
7. **Question Management:** Allows authorized administrators to securely view, add, edit, and delete educational questions.

Detailed functional requirements and expected system behavior are defined in Section 3.

### 2.3 User Characteristics

The Educational Ludo Game supports two main user types, each with different responsibilities and access permissions.

**1. Player:** The primary users are students studying Software Engineering and Software Quality Assurance. Players can configure and participate in games, roll dice, move game pieces, answer educational questions, and view game history. Players are not required to have advanced technical knowledge, and the interface should be understandable even to users unfamiliar with Ludo rules.

**2. Administrator:** An authorized user responsible for managing the educational question bank. Administrators can view, add, edit, and delete questions, including their difficulty levels and correct answers. Access to these functions must be restricted to authorized administrators to prevent unauthorized modification or exposure of the question database.

The system must provide an appropriate interface for each user type while maintaining the required access restrictions.

### 2.4 Constraints

The development of the Educational Ludo Game is subject to the following constraints defined in the official project specification:

**1. Programming Language:** The system must be implemented using Java, as required by the course staff.

**2. Software Architecture:** The application must follow the Model-View-Controller (MVC) architectural pattern and incorporate appropriate Design Patterns to support a modular and maintainable system structure.

**3. Data Storage:** Game history must be stored according to the predefined CSV schema supplied by the course staff. The required field names and their order must be preserved.

**4. Security and Access Control:** The question management interface must be accessible only to authorized administrators. The development team must select and justify an appropriate security mechanism to prevent unauthorized access to questions and correct answers.

**5. Gameplay Rules:** The system must comply with the official Ludo project specification, including the defined difficulty levels, board configurations, dice behavior, movement rules, and special square effects.

These constraints must be considered throughout system design and implementation. Their detailed requirements are addressed in Section 3.

### 2.5 Assumptions and Dependencies

The following assumptions and dependencies are considered during the specification and development of the Educational Ludo Game.

**Assumptions:**

1. **Local Gameplay:** The game is assumed to run on a single computer, with all players participating through the same interface. Online multiplayer functionality is not included in the current scope.

2. **Player Turn Order:** Unless otherwise specified, players are assumed to take turns according to the order in which they were entered during game setup.
3. **Game Timer:** The time displayed during gameplay is assumed to represent the elapsed time since the game started.
4. **Interrupted Games:** Only completed games are assumed to be recorded in game history. The handling of interrupted games remains a team decision.

**Dependencies:**

1. **Java Runtime:** The application depends on a compatible Java execution environment.
2. **Question Bank:** The educational functionality depends on the availability of at least 30 valid multiple-choice questions covering the three difficulty levels.
3. **Data Storage:** Game history depends on access to the required CSV file structure and appropriate file read/write permissions.
4. **Official Specifications:** The implementation depends on the official Ludo project requirements and clarifications provided by the course staff.

These assumptions represent the current specification baseline and will be reviewed if additional requirements or clarifications are provided by the course staff.

### 2.6 Apportioning of Requirements

The current version of the Educational Ludo Game is intended to implement all mandatory requirements defined in the official project specification, including gameplay functionality, difficulty levels, educational questions, game history, and administrator question management.

No requirements have been explicitly assigned by the course staff to future releases. Additional features, such as online multiplayer, computer-controlled opponents, or advanced player statistics, may be considered for future versions but are not part of the current system requirements.